
xmmPipeline

Suyog

Feb 13, 2023

CONTENTS:

1	xmmPipeline	1
1.1	xmmPipeline package	1
2	Indices and tables	17
	Python Module Index	19
	Index	21

XMMPIPELINE

1.1 xmmPipeline package

1.1.1 Submodules

1.1.2 xmmPipeline.convert module

For conversion of the RA and DEC.

1.1.3 xmmPipeline.epicObj module

10th May 2021: — Am making several changes to the code procedure arrangements. See the GitHub repo, the Notes on Google Docs and the documentation for more information. For previous docstring comments, see earlier code files, namely *xmmPipeline.py*

18th and 19th May 2021: — Adding options for Pile-up obsIDs.

21st May 2021: — Editing the *removeFlareBackground* function. A separate function *extract_flareGTI* will also be added.

9th~13th June 2021: — Starting to write the functions for obtaining individual background circles.

20th Jan 2023: — Making the code compatible with Mac. If the path names are Windows like, containing white-spaces, then we need to have additional single quotes around them before calling the *subprocess.run* command.

```
class xmmPipeline.epicObj.epicObj(ra, dec, workdir, sas_dir, headas, sas_ccfpath, srcCircRadius=None,  
                                  bkgCircRadius=None, dSrcThreshold=70, bkgCircGap=2.5,  
                                  edetectmode='chain', esp_nsplinenodes=14, gti_indiThreshold=500,  
                                  gti_combThreshold=None, lcBinSize=25, binBkgIc='no', saveFig=True,  
                                  showFig=True, ignorePileup=False)
```

Bases: `object`

combineEPICdata()

Combines EPIC PN, MOS1 and MOS2 Event lists data corresponding to the CCD where the Source is located.

Input

EPIC concatenated and calibrated Event lists obtained by running *emproc* and *epproc*.

Output

Combined PNMOS12.evts and MOS12.evts Event lists.

combineGTIs()

Combines Flare and Instrumental GTIs.

Input

Flare, EPIC PN, MOS1 and MOS2 GTIs.

Output

Combined Flare and Instrumental GTI FITS file.

downloadData()**Input**

List of obsIDs for which data has to be downloaded.

Output

None.

First it is checked is the the downloading has been done before. If yes, then if the size of the obsID folder is large, it's downloading is skipped.

extract_InstrumentalGTIs()

Extracts Instrumental GTIs from EPIC data using 'fextract' command.

Input

EPIC concatenated and calibrated Event lists obtained by running emproc and epproc.

Output

Intrumental GTI FITS files for EPIC PN, MOS1 and MOS2.

'fextract' command can't overwrite the extracted file if it already exists in the work directory, so they are deleted using 'rm -rf', which doesn't raises any error even if no such files are found for deleting during the first run.

extract_flareGTI()

Runs the XMMSAS commands to extract Flare GTI from PN Event List.

Input

Unfiltered EPIC PN and MOS1&2 Events lists.

Output

flareGTI file for the obsIDs.

This function largely follows the following SAS thread: <https://www.cosmos.esa.int/web/xmm-newton/sas-thread-epic-filterbackground>

Only one of either EPIC PN or MOS Flare Background filtering is required since the Flare affects the whole telescope.

extract_srcBkg_eventLists()

Extracts Source and Background Event list from the combined PNMOS12 data file, using the Source and Background location parameters given as Input. These Event lists will then be used for getting source images in JPEG using DS9.

19th May 2021: Added options for Pile-up obsIDs.

Input

Combined PNMOS12 Event list, Source and Background X, Y in Sky coords and rIn, rOut, the radii of the circles.

Output

Extracted Source and Background Event lists.

findObsIDs()**Input**

Coordinates of the object and path to work directory.

Output

List of obsIDs.

findSourceCCD()

Uses 'ecoordconv' command of SAS to find which CCD contains the Source in each of the EPIC's three cameras.

Input

EPIC concatenated and calibrated Event lists obtained by running emproc and epproc.

Output

None.

self.sourceCCDs is updated as a dictionary of dictionaries containing the Source CCD number for PN, MOS1 and MOS2 cameras for all the obsIDs.

find_otherSources()

Function to find all the other Sources, in addition to the main Source, lying within the region of interest. See: <https://www.cosmos.esa.int/web/xmm-newton/sas-thread-src-find-stepbystep>

Input

PNMOS12.evts and PN_CCD##.evts and corresponding extracted images in the Energy band 0.3-10 KeV using evselect.

Output

Exposer maps, masks, the csv file containing the x and y coordinates of the other Sources, apart from the region file from DS9.

Note that other sources may be found within any of the regions using different input files, for instance 'PN_image_full.fits' obtained from 'PNclean.ds' etc.

However, only the sources within the PNMOS12 overlap region or the PN region, in case of small mode, are of interest. Since, the PN region necessarily contains the overlap region within it regardless of the mode of observation, this should have been sufficient to get other sources. However, the MOS data adds some of its own sources and 'edetect_chain' is easier to use and can be used with multiple imagesets, so both 'PNMOS12' and 'PN_CCD##' are being used to get the sources.

Further, right now the SAS command 'edetect_chain', which is pretty slow when number of sources is large, is being used. Individual tasks within 'edetect_chain' can be run separately to fasten this up, although that would require changes in the createOutputImages() function below.

find_otherSources_indi(inst='PN')

Function to find all the other Sources, in addition to the main Source, lying within the region of interest. See: <https://www.cosmos.esa.int/web/xmm-newton/sas-thread-src-find-stepbystep>

Parameters

inst (*str*) – name of the instrument of use.

Input

inst_CCD##.evts Event list.

Raises

Error – if *inst* is not in ['PN', 'MOS1', 'MOS2']

Output

Exposer maps, masks, the csv file containing the x and y coordinates of the other Sources,

apart from the region file from DS9. Also extracts *inst_image_CCD##.fits* file which is later used to find srcBkg circls. Updates the `otherSourceIndi` dictionary for each of the obsIDs in `self.obsIDs`

Notes

This separate function based on *find_otherSources* is meant to be used for finding individual instrument other sources. It uses `edetect_chain` function from XMMSAS for detecting the Sources. *inst_CCD##.evts* is required to extract corresponding images in the Energy band 0.3-10 KeV using `evselect`.

- The indi inst files have `inst` appended at the end of their names.
- *emllist.fits* originally found for the overlap is rewritten with every run of `edetect_chain`. However, the *emllist.csv* is not b'cuz of the above point So *emllist.csv* should be used for the overlap cases. And anyway, another run of the original *find_otherSources* will update the said file.

Todo: Check is using `PATTERN` in `[0:12]` in `evselect` expression for PN is correct? For data extraction elsewhere, the expression for PN reads, (`PATTERN <= 4`)

`getBackgroundCircles()`

Calls *findOverlap.py* functions to automatically detect the overlap PNMOS12 region and find 2 background circles in it. If the MOS mode is *small*, then the PN image is used for getting the background circles.

Input

PN_image.fits, MOS12_image.fits and PNMOS12_image.fits to get the overlap from.

Output

x, y coords of the background circles alongwith the radii of these circles and the source circle. Units of radii are pixels or Sky Coord units. Updates the `sourceLoc` and `backgroundLoc` dictionaries with the location parameters.

`getBackgroundCircles_indi (inst='PN')`

Calls the *backgroundCircles* function in *findOverlap.py* to automatically find 2 background circles in the data of the instrument passed.

Parameters

inst (*str*) – name of the instrument of use.

Input

inst_CCD##_image.fits files containing the extracted image data of the instrument.

Requires

Call to *find_otherSources_indi* function for the dictionary of other Sources.

Raises

Error – if *inst* is not in ['PN', 'MOS1', 'MOS2']

Output

x, y coords of the background circles alongwith the radii of these circles. Units of radii are pixels or Sky Coord units. Updates the `backgroundLocIndi` dictionaries with the background location parameters.

Notes

small-mode MOS checking is invalid here since overlap is not been detected. The CCD argument in `findOverlap` class is named `pnCCD`.

obtain_lightCurves()

Obtains the light curves from the Source and Background Event lists.

Input

Source and Background Event lists.

Output

Corresponding light curves.

By default, the background light curve is not binned.

readCCDcoordsPickle()

Reads `ccd_coords_info.pickle` file which contains the location parameters.

reduceEPICdata()

Runs the XMMSAS commands on the shell to reduce EPIC data.

Input

SAS Summary file in the work directory, that is downloaded alongwith the data.

Output

EPIC PN and MOS1&2 calibrated and concatenated Event lists produced by SAS tasks *epproc* and *emproc*.

These event lists contain Instrumental GTI for each of the CCDs. See Notes and <http://xmm-tools.cosmos.esa.int/external/sas/current/doc/epicproc/node17.html>

This function largely follows the following SAS thread: <https://www.cosmos.esa.int/web/xmm-newton/sas-thread-epic-reprocessing>

removeFlareBackground()

Runs the XMMSAS commands to remove Flare background from EPIC Event List and obtained cleaned event lists for each EPIC instrument. The Flare GTI file extracted previously using *extract_flareGTI* is used. Since the Flare Background is same for all the instruments onboard, this same *flareGTI* file can be used for them all.

Note that these Flare Background filtered Event Lists are only used for Spectral Analysis, since for Light Curve extraction, *combinedGTI* file and combined *PNMOS12.evts* EPIC data is used.

Input

Unfiltered EPIC PN and MOS1&2 Events lists and *flareGTI* file.

Output

Flare Background filtered Event Lists for each EPIC instrument.

This function largely follows the following SAS thread: <https://www.cosmos.esa.int/web/xmm-newton/sas-thread-epic-filterbackground>

save_results()

Does two tasks,

One, saves a pickle file at each obsID directory containing

sourceCCDs, sourceLoc, backgroundLoc and otherSources for it.

Two, copies the Source, Background Events lists and other output files

from each obsID directory to a results folder in the main directory.

sortObsIDs(*objName=None*)

To sort the obsIDs in *obsIDs_list.dat* by reading it into a pandas dataframe.

updateCCDcoordsPickle(*inst=None*)

Writes the background location parameter dictionary for individual instruments *backgroundLoc_indi* to *ccd_coords_info.pickle* file. If the file is already present in the directory, then the values for the corresponding keys are updated.

Parameters

inst (*str*) – name of the instrument of use.

Input

backgroundLoc_indi dictionary containing the location parameters.

Output

Updated *ccd_coords_info.pickle* file.

Notes

If no CCDcoordsPickle file is available in the *workdir*, then the whole file is created with other values been saved as well.

writeCCDcoordsPickle()

Writes the location parameters to *ccd_coords_info.pickle* file. If the file is already present in the directory, then the values for the corresponding keys will be updated.

21st May 2021: Added loading *ccd_coords_info.pickle* file if it is already present in the directory.

`xmmPipeline.epicObj.printErrorMessage(message)`

`xmmPipeline.epicObj.strToBool(s, inverse=False)`

1.1.4 xmmPipeline.epicPileup module

08 May 2021: — For clearing the EPIC Pile-up in early time obsIDs. See: <https://www.cosmos.esa.int/web/xmm-newton/sas-thread-epatplot>

This would be used to correct for the Pile-up separately for individual obsIDs. Thus, the *workdir* is changed to `'workdir'+'/work'` folder.

This procedure will give the Source Annulus radii for which Pile-up is negligible for any particular obsIDs. Henceforth, it is this Source region for which the Light Curve and the Spectra extraction should be done.

So, I will have to:

- > **Modify *xmmPipeline.py* so that it knows which obsIDs have the Pile-up issue and need the Source region to lie within an Annulus.**
- > **Modify *spectra.py* so that the Source Spectra are extracted from within this Annulus.**

See the notes of Google Doc for open questions.

09 May 2021: — *epatplot* calculates two diagnostic numbers which may be used to assess the presence of pile-up: In the absence of pile-up, the 0.5 - 2.0 keV (default range) observed-to-model singles and doubles pattern fractions ratios should both be consistent with 1.0 within statistical errors (1 sigma errors are given). If pile-up is present, the singles ratio will be smaller than 1.0 and the doubles ratio will be larger than 1.0.

13 May 2021: — Adding *correctPileUp* function. Some late-time obsIDs 0770981001, 0810200501 and 0810200701 also appear to be Piled-up! Gotta get this discrepancy cleared up.

```
class xmmPipeline.epicPileup.epicPileup(ra, dec, workdir, sas_dir, headas, sas_ccfpath,  
                                         srcCircRadius=None, bkgCircRadius=None, dSrcThreshold=70,  
                                         bkgCircGap=2.5, edetectmode='chain', esp_nsplinenodes=14,  
                                         gti_indiThreshold=500, gti_combThreshold=None,  
                                         lcBinSize=25, binBkgIc='no', saveFig=True, showFig=True,  
                                         ignorePileup=False)
```

Bases: `epicObj`

checkPileUp()

Uses *epatplot* to check for the Pile-up issue.

First a GTI filtered Source Event list is extracted out of *PN_CCD#.evts* and then *epatplot* is run to check the observed Spectra with the expected pattern distribution function.

epatplot calculates two diagnostic numbers which may be used to assess the presence of pile-up: In the absence of pile-up, the 0.5 - 2.0 keV (default) observed-to-model singles and doubles pattern fractions ratios should both be consistent with 1.0 within statistical errors (1 sigma errors are given). If pile-up is present, the singles ratio will be smaller than 1.0 and the doubles ratio will be larger than 1.0. See: <https://www.cosmos.esa.int/web/xmm-newton/sas-thread-epatplot>

Note that *pnGTI.fits* GTI file is used instead of *combinedGTI_obsID.fits* because the latter results in very few useful data points.

If the obsID is found to be Piled-up then the user should run the *correctPileUp* function to correct for it.

Input

Concatenated and Calibrated Event list, pnGTI file, *sourceCCDs* and the Source location parameters.

Output

The Source Annulus radii together with Filtered Event List *source_PN_filtered_obsID.evts* and the corresponding image *source_PN_filteredPattern.ps*.

Note

The term *filtered* means that the GTI have been applied.

correctPileUp(*srcRin=None*)

Uses *epatplot* to correct and check for the Pile-up issue by removing some pixels from the center of the Source circle.

The observation is corrected for Pile-up by removing some pixels from the center of the Source. Now, this checking for Pile-up need be done only once, for PN images, since, it will be the same for MOS, probably.

So, if Pile-up is found, one has to manually check for by iteratively removing more and more center pixels and find when the Pile-up becomes negligible. Then the Source circle would be an Annulus for this region. The same Annulus parameters can be repeated for MOS1&2. See: <https://www.cosmos.esa.int/web/xmm-newton/sas-thread-epatplot>

Note that *pnGTI.fits* GTI file is used instead of *combinedGTI_obsID.fits* because the latter results in very few useful data points.

The result of this procedure would be the radii values of the Source Annulus for which Pile-up is negligible. These new found *rIn* and *rOut* are then saved into to the *epicObj* and are written to *ccd_coords_info.pickle* file. Henceforth, for these Pile-up obsIDs the Source region defined from *rIn* to *rOut* will be used for obtaining the Spectra.

Input

Concatenated and Calibrated Event list, pnGTI file, *sourceCCDs* and the Source location parameters.

Output

The Source Annulus radii together with Filtered Event List *source_PN_filtered_obsID.evts* and the corresponding image *source_PN_filteredPattern.ps*, and the Annulus files *source_PN_filteredAnnulus_obsID.evts* and *source_PN_filteredPattern_Annulus.ps* showing resolution of the Pile-up issue.

Note

The term *filtered* means that the GTI have been applied.

save_pileUpResults()

Saves the **filteredPattern.ps* and **filteredAnnulus*.ps* to the results directory.

`xmmPipeline.epicPileup.main(args)`

1.1.5 xmmPipeline.epicPipeline module

10th May 2021: — Am making several changes to the code procedure arrangements. See the GitHub repo, the Notes on Google Docs and the documentation for more information. For previous docstring comments, see earlier code files, namely *xmmPipeline.py*

`xmmPipeline.epicPipeline.combineAndFind_method(args)`

`xmmPipeline.epicPipeline.extractProds_method(args)`

`xmmPipeline.epicPipeline.reduceData_method(args)`

`xmmPipeline.epicPipeline.runIndiBkgCircFuncs_method(args)`

1.1.6 xmmPipeline.epicSpectra module

10th May 2021: — Am making several changes to the code procedure arrangements. See the GitHub repo, the Notes on Google Docs and the documentation for more information. For previous docstring comments, see earlier code files, namely *spectra.py*

11th May 2021: — Since model fitting for individual obsIDs has to be done manually, I don't think *xspect_fitSpectra* would be used now. *pyXspec.py* will have to be run manually for each obsIDs with the different model parameters varying.

20th May 2021: — Completing the pending work of fitting Models to the Spectra. Two criterions to be checked for each obsIDs:

- whether obsID is Piled-up? DONE!
- whether obsID is in Small-mode? DONE!

21st May 2021: — Changed to using Flare Background filtered Event Lists for Spectra extraction. Found a way to save the output parameter values from *pyXspec.py*. Completing the SpecModel fitting now.

13th June 2021: — Completing the work for obtaining individual instrument bkgCircs.

- Adding *runIndiBkgCircFuncs* to the main function.

17th June 2021: — Adding features to read individual instrument bkgCircs from *ccdCoordsPickle* file. And finally starting the manual specModel fits.

```
class xmmPipeline.epicSpectra.epicSpectra(doNotOverwriteModel=False, *args, **kwargs)
```

Bases: epicObj

```
extractSpectra_imageMode()
```

Extracts the MOS12 and PN spectra in Image Mode. See: <https://www.cosmos.esa.int/web/xmm-newton/sas-thread-mos-spectrum>,

<https://www.cosmos.esa.int/web/xmm-newton/sas-thread-pn-spectrum>

Input

Flared Background filtered PN, MOS1&2 Event Lists, *PNclean.ds*, *MOS1clean.ds* and *MOS2clean.ds*. Alongwith SrcBkg coordinates and SourceCCDs resulting from *epicPipeline.py*

Output

MOS12 and PN SrcBkg Spectra in Image Mode, a Redistribution Matrix (*rmf*) file and an Effective Area Vector (*arf*) file.

Notes

Note that for Small Mode obsIDs, since the background is taken from PN image data, instead of the overlap region data b'cuz MOS12 region was of small size, the background region will lie outside of the image. Thus, only the Source Spectra can possibly be extracted for Small Mode obsIDs. However, the 'specgroup' command requires both the Source and the Background Spectra in order to create the grouped Spectrum. Therefore, as of now, I am completely skipping getting the MOS12 Spectra for Small mode obsIDs.

20th May 2021: Now works with Pile-up corrected obsIDs. Separate Source Spectrum file is generated for Piled-up cases, *inst_spectrum_source_annulus_obsID.fits* All the other file names, including the Grouped Spectra is not altered.

17th June 2021: Adding feature for individual instrument bkgCircs.

If the total area of indi inst bkgCircs is greater than that of the overlap bkgCircs, or if the former is not available for any given obsID, the overlap bkgCircs are used.

```
readUpdatedCCDcoordsPickle()
```

Reads the individual instrument bkgCircs keywords from the updated *ccd_coords_info.pickle* file which contains the location parameters.

```
save_spectraResults()
```

Does one tasks

Copies the Spectra images from each obsID directory to a results folder in the main directory.

```
xspec_fitSpectra(instName='PN')
```

Fits a model Spectra to the extracted Spectra data using *pyXspec* package. Since, *pyXspec* requires HEA initialisation, a separate routine *pyXspec.py* is required to run it. See: <https://www.cosmos.esa.int/web/xmm-newton/sas-thread-xspec>

Input

Extracted Spectra FITS file and input model parameters.

Output

Spectra plots and *specModelParams.json* file.

20th May 2021: Adding feature to allow model parameter value inputs.

```
xmmPipeline.epicSpectra.main(args)
```

1.1.7 xmmPipeline.findOverlap module

08-12 February 2021: — New Python function to obtain the overlap region.

22 February 2021: — Completing the modifications to the codes that were discussed on 13th of this month.

26 February 2021: — Had the call with Dheeraj@MIT yester. Finalizing the things about this pipeline now.

10 March 2021: — The radius of the Source and Background circles given as input to `evselect` have to be in Sky Coords or Pixel units and not arcsec, as I had been giving. Thus, the returned values from `findOverlap.py` have to be altered.

30 March 2021: — Was thinking of using `numba` to accelerate the code, but since most of the functions use `matplotlib` it is perhaps unfeasible.

29th April 2021: — The Background circles were too small in some cases. Modifying the criterion for background circles radius and threshold.

Instead of taking some random point from with the points that are a threshold distance away from other Sources, I can take the one with max radii. Currently, the Bkg radius is defined a-priori.

Tried using this, but there are so many complications. So, have simply added another function to find the max bkg radii possible within the threshold, to be used after the bkg coords have been found.

I think, I can have `Bx1,By1` be found together with the `xUseN,yUseN` and continue having `__maxBkgRadius` function for the Radii. Yeah! This works. However, whether or not optimum radii are obtained, depends on which of the circles is taken as first and which as the second.

I can also try to have the `xmmObj` class as a meta-class for the `findOverlap` class. Nope! It's better not to do this because I run `findOverlap.py` for each obsID separately.

30th April 2021: — Made the second background point non-random as well. The program takes a little longer now, but both the circles are large sized. Gonna check if there's any errors that come up because of this. Also lowered the Bkg Circle Gap to 2.5 pixels instead of 5 pixels.

13th June 2021: — Writing the functions for obtaining individual background circles. Learned about this awesome algorithm, `PolyLabel` for finding the *pole of inaccessibility* or the point of largest inscribed circle for any polygon. My problem of finding the Bkg circles is exactly the same. See: <https://github.com/Twista/python-polylabel>

Damn! That `polylabel` requires the boundary of the polygon to be defined and hell, I have the whole points in the region. Running the `polylabel(points)` on that full throttles the system. Hell! My own solution is pretty elegant then. Better suited for my purpose.

Wait a minute, what I can do to find more number of Bkg points is first find the first point and then remove all the points lying in the 2R distance from the center of this first Bkg point and repeat. Let's try this!

Yosh! This works brilliantly. Both the BkgPts are now randomly generated in the given area satisfying the conditions. Now, I can theoretically iterate a few times until I get both the BkgCircs of sufficiently useful area. However, what my aim has been ever since the start was to have the algorithm automatically find two of the largest BkgCircs satisfying the conditions. Hmm... I think that would be a pretty daunting task and a difficult one at that, since I haven't go just what I have been wanting since a long time now.

What is much better in my opinion is that I can repeat the procedure of filtering out the last found BkgPt + 2R dist points n number of times to obtain n number of Bkg points. Let's try that out now!

This recursive way of finding the background circles takes care of overlap between the circles all by itself, since the `xToUse` and `yToUse` are recursively filtered for the circle area. Yup! Have verified that most of the times, this recursive way works! Woohooo yayyy! Such a great thing. For the times when it doesn't work, one of the bkgCirc overlaps with another bkgCircs, and consquensly the radius value for comes out to be negative. I am adding feature to raise a warning when this happens. A simple rerun of the code will remove the problem and yield correct bkgCircs, although yeah! I may need to check this up with all the obsIDs. And it would be better to have the user input the `nBkgCirc` value, so that if it's not possible to obtain more than 2 bkgCircs, the user can find 2 bkgCircs atleast.

```
class xmmPipeline.findOverlap.findOverlap(workdir, obsID='NA', pnCCD='4', srcCoords=None,
                                          otherSrc=None, gap=2.5, srcR=None, bkgR=None,
                                          srcThreshold=None, saveFig=True, showFig=True)
```

Bases: `object`

backgroundCircles(*axes*)

Input

The output from ‘`obtainOverlap()`’ function aka detected overlap region., and the Source coordinates.

Output

Coordinates and Radii of the Background Circles.

backgroundCircles_indi(*ax*, *inst*, *nBkgCircs*=3)

Automatically finds Background Circles in the instrument data passed. Modified the function a great deal from the earlier naive *backgroundCircles* function.

Input

inst_CCD##_image.fits files containing the extracted image data of the instrument.
srcCoords dictionary containing coords of the main Source. *otherSrc* dictionary containing coords of other Sources.

Requires

Call to *epicObj.find_otherSources_indi* function.

Parameters

- **ax** (*obj*) – *matplotlib.pyplot.subplots* object for plotting the images.
- **inst** (*str*) – name of the instrument of use.
- **nBkgCircs** (*int*) – number of background circles to detect. Defaults to 3. (Optional)

Output

Coordinates and Radii of the Background Circles.

Notes

small-mode MOS checking is invalid here since overlap is not been detected.

createOutputImages(*inst*=None)

Creates the *source_background_circles.png* output image showing the other Sources and the Source and Background circles overlayed on top of the image. The overlap (‘PNMOS12_image_full’ for Small-Mode cases) is used as the backdrop for cases when the overlap detection is performed and the circles are found using that overlap data. Otherwise, *inst_CCD##_image* is used as the backdrop.

Parameters

inst (*str*) – Name of the instrument to use. Defaults to None. If None then it is assumed at the overlap detected is performed.

Input

findOverlap object with all previous functions already run.

Output

Output image saved in the *workdir* of the *obsID*. Image file name is *source_background_circles.png* for the default case and *srcBkg_circs_inst.png* for when individual instrument background circles are been found.

main(*inst=None*)

The main function to sequentially run the functions in *findOverlap* class.

Returns

(x, y, r) coords and radius of first background circle. bCircle2 (tuple): (x, y, r) coords and radius of second background circle.

correctSrc (tuple): corrected (x, y) coords of the main Source.

The original main Source coords found by the *epicObj.findSourceCCD* function using the XMMSAS *ecoordconv* utility are a little bit shifted from what the main Source likely is as found from *edetect_chain* while finding other Sources. So, the main Source coords are changed to the coords of the nearest Source to it in the *otherSrc* dictionary.

srcR (float): radius of the Source circle or outer radius of the Source Annulus

for the Piled-up cases, although a full circle is detected here.

isSmallMode (bool): True, if absolute difference between flattened MOS12 and

PNMOS12 image arrays is less than 500. False otherwise. See issue #28 in the xmmPipeline repository in this regard, <https://github.com/Suyog7130/xmmPipeline/issues/28>

Return type

bCircle1 (tuple)

Notes

All returned coordinates are in **Physical Units**. These (x, y) are termed as (**RAWX**, **RAWY**) in DS9. The radius values are in **Sky Coordinates**, which I think is the name XMMSAS gives to **Physical Units**. Each Sky Coord unit equals **0.05 arcsec**. So, a radius of 640 equals 32 arcsec. See issue #24 for more details, <https://github.com/Suyog7130/xmmPipeline/issues/24>

obtainOverlap(*axes*)

Input

The workdir where the PN, MOS and PNMOS12 FITS files are located.

Output

The detected overlap region if it is larger in size than then MOS region, or else the PNMOS12 image data.

xmmPipeline.findOverlap.**imageMOS12**(*workdir*)

xmmPipeline.findOverlap.**imageMOS12_soft**(*workdir*)

xmmPipeline.findOverlap.**imagePN**(*workdir*, *pnCCD*)

xmmPipeline.findOverlap.**imagePNMOS12**(*workdir*)

xmmPipeline.findOverlap.**imagePNMOS12_soft**(*workdir*)

xmmPipeline.findOverlap.**imagePN_soft**(*workdir*, *pnCCD*)

1.1.8 xmmPipeline.getSpectraFitsTable module

22nd May 2021: — Have completed much of the pending work regarding Spectral Analysis of ASASSN-14li. This Python routine is to read the *specModelParams.json* files for each of the obsIDs and create a *specResultsTable* using them.

I don't the obsIDs thing will be valid here. Why would one not wanna include some obsIDs in the final table? Anyway.

23rd May 2021: — The primary for loop should be for model since each model will have a separate *specResultsTable*.

18th June 2021: — Finishing up the manual *specModel* fitting.

NOTE: Be wary of the *specModel* name with single additive model component when it has parentheses around it. The model name saved in the JSON file and that been given as input, should exactly match. See issue #31 in regard to this. <https://github.com/Suyog7130/xmmPipeline/issues/31>

Swaping the Row of the *specResultsTable* with the columns will take sometime.

```
xmmPipeline.getSpectraFitsTable.main(args)
```

1.1.9 xmmPipeline.plotAnal module

13th May 2020: A module to ease up beatifying the plots.

13th June 2020: Made changes on this. If the fit is linear regression then the rSquared coefficient of determination is the square of the pearson correlation coefficient.

20th August 2020: Adding keyword for number of ticks shown.

23rd September 2020: Adding keyword for axis colours via `ax.spines['bottom'].set_color()`

NOTE: spines can only be used with figure and not subplots!

24th January 2021: Made modifications in the `beautifyPlot()`.

14th April 2021: — Adding modifications to the `beautifyPlot` function.

```
class xmmPipeline.plotAnal.plotAnal(figure, x, y)
```

Bases: `object`

```
PSNR(x, y, xloc=0.05, yloc=0.1, decPlace=2, fontsize=15)
```

Calculates Peak Signal-to-Noise Ratio between x and y and plots its value on the figure.

```
SSIM(x, y, xloc=0.05, yloc=0.1, decPlace=2, fontsize=15, scikit_value=False)
```

Calculates the Structural SIMilarity index between two image data given by x and y. The calculated value is then plotted on the figure passed.

```
SSIM_value(y, decPlace=2, scikit_value=False)
```

Calculates the Structural SIMilarity index between two image data given by x and y.

```
beautifyPlot(labelsize=11, lengthMajor=10, tickNum=10, tickDirection='out', lengthMinor=5,  
             minor=False, grid=False, axisColor=None, yTicks=True, xTicks=True, logFormat=None,  
             logXformat=None, logYformat=None, logMinor=True, minorLabel=False,  
             logXminorLabel=False, logYminorLabel=False)
```

If axes subplots is used, with multiple rows and columns, then the ndarray of axes subobjects can be flattened:

```
axes.flatten().tolist()
```

The axisColor takes in a dictionary axisColor with keys as the spine position and values as the colour that has to be inserted. Be wary of the american spelling of ‘colour’, ‘color’, used in here. NOTE: spines can only be used with figure and not subplots!

```
linearFit(x, y, lineDetails=True, xloc=0.05, yloc=0.1, fontsize=15, color='darkorange')
```

```
linearFit_plotCal(x, y, xloc=0.05, yloc=0.15, fontsize=15)
```

```
pearson(x, y, line=False, xloc=0.05, yloc=0.1, decPlace=2, rSquared=False, fontsize=15,  
        linecolor='darkorange', lineDetails=False)
```

```
pearson_spearman(x, y, line=False, xloc=0.05, yloc=0.1, decPlace=2, rSquared=False, fontsize=15,  
                  linecolor='darkorange', lineDetails=False)
```

```
rSquared(y, f)
```

This can only be there between the y values and the fitted values f from the fitted line.

```
spearman(x, y, line=False, xloc=0.05, yloc=0.1, decPlace=2, rSquared=False, fontsize=15,  
          linecolor='darkorange', lineDetails=False)
```

1.1.10 xmmPipeline.pyXspec module

1.1.11 xmmPipeline.rgsPipeline module

01 April 2021: — Python routine to reduce the RGS data, obtain the Spectra and the light curve from it. The SAS threads to be used, in order of execution are:

1. <https://www.cosmos.esa.int/web/xmm-newton/sas-thread-rgs> Main thread to reduce the RGS data. Similar to ‘emproc’ and ‘epproc’. Should do all the work and obtain the Spectra and the light curves.
2. <https://www.cosmos.esa.int/web/xmm-newton/sas-thread-rgs2> To correct the Source coordinates and the extraction masks, if the above step was incorrect.
3. <https://www.cosmos.esa.int/web/xmm-newton/sas-thread-timing> For extraction of RGS light curves.

```
xmmPipeline.rgsPipeline.main(args)
```

```
xmmPipeline.rgsPipeline.printErrorMessage(message)
```

```
class xmmPipeline.rgsPipeline.rgsObj(ra, dec, workdir, sas_dir, headas, sas_ccfpath)
```

Bases: `object`

```
findObsIDs()
```

Input

Coordinates of the object and path to work directory.

Output

List of obsIDs.

```
readPickleFile()
```

Reads the already obtained pickle file containing location parameters.

```
reduceRGSdata()
```

Reduces the RGS data using ‘rgsproc’ to obtain Spectra. A folder names ‘RGS/’ is created in the work folder of each obsID directory. See: <https://www.cosmos.esa.int/web/xmm-newton/sas-thread-rgs>

Input

None.

Output

Source and Background RGS Spectra and the associated response matrices.

save_results()

Does two tasks,

One, saves a pickle file at each obsID directory containing

sourceCCDs, sourceLoc, backgroundLoc and otherSources for it.

Two, copies the Source, Background Events lists and other output files

from each obsID directory to a results folder in the main directory.

`xmmPipeline.rgsPipeline.strToBool(s)`

1.1.12 Module contents

INDICES AND TABLES

- `genindex`
- `modindex`
- `search`

PYTHON MODULE INDEX

X

- `xmmPipeline`, [15](#)
- `xmmPipeline.convert`, [1](#)
- `xmmPipeline.epicObj`, [1](#)
- `xmmPipeline.epicPileup`, [6](#)
- `xmmPipeline.epicPipeline`, [8](#)
- `xmmPipeline.epicSpectra`, [8](#)
- `xmmPipeline.findOverlap`, [10](#)
- `xmmPipeline.getSpectraFitsTable`, [13](#)
- `xmmPipeline.plotAnal`, [13](#)
- `xmmPipeline.rgsPipeline`, [14](#)

B

`backgroundCircles()`
(*xmmPipeline.findOverlap.findOverlap*
method), 11

`backgroundCircles_indi()`
(*xmmPipeline.findOverlap.findOverlap*
method), 11

`beautifyPlot()` (*xmmPipeline.plotAnal.plotAnal*
method), 13

C

`checkPileUp()` (*xmmPipeline.epicPileup.epicPileup*
method), 7

`combineAndFind_method()` (in module
xmmPipeline.epicPipeline), 8

`combineEPICdata()` (*xmmPipeline.epicObj.epicObj*
method), 1

`combineGTIs()` (*xmmPipeline.epicObj.epicObj*
method), 1

`correctPileUp()` (*xmmPipeline.epicPileup.epicPileup*
method), 7

`createOutputImages()`
(*xmmPipeline.findOverlap.findOverlap*
method), 11

D

`downloadData()` (*xmmPipeline.epicObj.epicObj*
method), 2

E

`epicObj` (class in *xmmPipeline.epicObj*), 1

`epicPileup` (class in *xmmPipeline.epicPileup*), 7

`epicSpectra` (class in *xmmPipeline.epicSpectra*), 8

`extract_flareGTI()` (*xmmPipeline.epicObj.epicObj*
method), 2

`extract_InstrumentalGTIs()`
(*xmmPipeline.epicObj.epicObj* *method*),
2

`extract_srcBkg_eventLists()`
(*xmmPipeline.epicObj.epicObj* *method*),
2

`extractProds_method()` (in module
xmmPipeline.epicPipeline), 8

`extractSpectra_imageMode()`
(*xmmPipeline.epicSpectra.epicSpectra*
method), 9

F

`find_otherSources()` (*xmmPipeline.epicObj.epicObj*
method), 3

`find_otherSources_indi()`
(*xmmPipeline.epicObj.epicObj* *method*),
3

`findObsIDs()` (*xmmPipeline.epicObj.epicObj* *method*),
2

`findObsIDs()` (*xmmPipeline.rgsPipeline.rgsObj*
method), 14

`findOverlap` (class in *xmmPipeline.findOverlap*), 10

`findSourceCCD()` (*xmmPipeline.epicObj.epicObj*
method), 3

G

`getBackgroundCircles()`
(*xmmPipeline.epicObj.epicObj* *method*),
4

`getBackgroundCircles_indi()`
(*xmmPipeline.epicObj.epicObj* *method*),
4

I

`imageMOS12()` (in module *xmmPipeline.findOverlap*), 12

`imageMOS12_soft()` (in module
xmmPipeline.findOverlap), 12

`imagePN()` (in module *xmmPipeline.findOverlap*), 12

`imagePN_soft()` (in module *xmmPipeline.findOverlap*),
12

`imagePNMOS12()` (in module *xmmPipeline.findOverlap*),
12

`imagePNMOS12_soft()` (in module
xmmPipeline.findOverlap), 12

L

`linearFit()` (*xmmPipeline.plotAnal.plotAnal* *method*),

14
linearFit_plotCal()
(xmmPipeline.plotAnal.plotAnal method),
14

M

main() (in module xmmPipeline.epicPileup), 8
main() (in module xmmPipeline.epicSpectra), 9
main() (in module xmmPipeline.getSpectraFitsTable), 13
main() (in module xmmPipeline.rgsPipeline), 14
main() (xmmPipeline.findOverlap.findOverlap method),
11

module

xmmPipeline, 15
xmmPipeline.convert, 1
xmmPipeline.epicObj, 1
xmmPipeline.epicPileup, 6
xmmPipeline.epicPipeline, 8
xmmPipeline.epicSpectra, 8
xmmPipeline.findOverlap, 10
xmmPipeline.getSpectraFitsTable, 13
xmmPipeline.plotAnal, 13
xmmPipeline.rgsPipeline, 14

O

obtain_lightCurves()
(xmmPipeline.epicObj.epicObj method),
5
obtainOverlap() (xmmPipeline.findOverlap.findOverlap
method), 12

P

pearson() (xmmPipeline.plotAnal.plotAnal method), 14
pearson_spearman() (xmmPipeline.plotAnal.plotAnal
method), 14
plotAnal (class in xmmPipeline.plotAnal), 13
printErrorMessage() (in module
xmmPipeline.epicObj), 6
printErrorMessage() (in module
xmmPipeline.rgsPipeline), 14
PSNR() (xmmPipeline.plotAnal.plotAnal method), 13

R

readCCDcoordsPickle()
(xmmPipeline.epicObj.epicObj method),
5
readPickleFile() (xmmPipeline.rgsPipeline.rgsObj
method), 14
readUpdatedCCDcoordsPickle()
(xmmPipeline.epicSpectra.epicSpectra
method), 9
reduceData_method() (in module
xmmPipeline.epicPipeline), 8

reduceEPICdata() (xmmPipeline.epicObj.epicObj
method), 5
reduceRGSdata() (xmmPipeline.rgsPipeline.rgsObj
method), 14
removeFlareBackground()
(xmmPipeline.epicObj.epicObj method),
5
rgsObj (class in xmmPipeline.rgsPipeline), 14
rSquared() (xmmPipeline.plotAnal.plotAnal method),
14
runIndiBkgCircFuncs_method() (in module
xmmPipeline.epicPipeline), 8

S

save_pileUpResults()
(xmmPipeline.epicPileup.epicPileup method),
8
save_results() (xmmPipeline.epicObj.epicObj
method), 5
save_results() (xmmPipeline.rgsPipeline.rgsObj
method), 15
save_spectraResults()
(xmmPipeline.epicSpectra.epicSpectra
method), 9
sortObsIDs() (xmmPipeline.epicObj.epicObj method),
5
spearman() (xmmPipeline.plotAnal.plotAnal method),
14
SSIM() (xmmPipeline.plotAnal.plotAnal method), 13
SSIM_value() (xmmPipeline.plotAnal.plotAnal
method), 13
strToBool() (in module xmmPipeline.epicObj), 6
strToBool() (in module xmmPipeline.rgsPipeline), 15

U

updateCCDcoordsPickle()
(xmmPipeline.epicObj.epicObj method),
6

W

writeCCDcoordsPickle()
(xmmPipeline.epicObj.epicObj method),
6

X

xmmPipeline
module, 15
xmmPipeline.convert
module, 1
xmmPipeline.epicObj
module, 1
xmmPipeline.epicPileup
module, 6
xmmPipeline.epicPipeline

- module, 8
- xmmPipeline.epicSpectra
 - module, 8
- xmmPipeline.findOverlap
 - module, 10
- xmmPipeline.getSpectraFitsTable
 - module, 13
- xmmPipeline.plotAnal
 - module, 13
- xmmPipeline.rgsPipeline
 - module, 14
- xspec_fitSpectra() (*xmmPipeline.epicSpectra.epicSpectra*
method), 9